



PDF Download
3585341.3585378.pdf
27 March 2026
Total Citations: 5
Total Downloads: 95

 Latest updates: <https://dl.acm.org/doi/10.1145/3585341.3585378>

RESEARCH-ARTICLE

Evaluation of SYCL's Suitability for High-Performance Critical Systems

CRISTINA QUESADA PERALTA, Barcelona Supercomputing Center, Barcelona, Barcelona, Spain

MATINA MARIA TROMPOUKI, Barcelona Supercomputing Center, Barcelona, Barcelona, Spain

LEONIDAS KOSMIDIS, Barcelona Supercomputing Center, Barcelona, Barcelona, Spain

Open Access Support provided by:

Barcelona Supercomputing Center

Published: 18 April 2023

[Citation in BibTeX format](#)

IWOCL '23: International Workshop on OpenCL

April 18 - 20, 2023

Cambridge, United Kingdom

Evaluation of SYCL's Suitability for High-Performance Critical Systems

Cristina Peralta Quesada
Universitat Politècnica de
Catalunya (UPC)
Barcelona Supercomputing
Center (BSC)
Spain

cristina.peralta.quesada@estudiantat.upc.edu

Matina Maria Trompouki
Barcelona Supercomputing
Center (BSC)
Universitat Politècnica de
Catalunya (UPC)
Spain

matina.trompouki@bsc.es

Leonidas Kosmidis
Barcelona Supercomputing
Center (BSC)
Universitat Politècnica de
Catalunya (UPC)
Spain

leonidas.kosmidis@bsc.es

ABSTRACT

Upcoming safety critical systems require high performance processing, which can be provided by multi-cores and embedded GPUs found in several Systems-on-chip (SoC) targeting these domains. So far, only low-level programming models and APIs, such as CUDA or OpenCL have been evaluated. In this paper, we evaluate the effectiveness of a higher level programming model, SYCL, for critical applications executed in such embedded platforms. In particular, we are interested in two aspects: performance and programmability.

In order to conduct our study, we use the open source GPU4S Bench benchmarking suite for space and an open source pedestrian detection application representing the automotive sector, which we port into SYCL and analyse their behavior. We perform our evaluation on a high-performance platform featuring an NVIDIA GTX 1080Ti as well as a representative embedded platform, the NVIDIA Xavier AGX which is considered a good candidate for future safety critical systems in both domains and we compare our results with other programming models. Our results show that in several cases SYCL is able to obtain performance close to highly optimised code using CUDA or NVIDIA libraries, with significantly lower development effort and complexity, which confirms the suitability of SYCL for programming high-performance safety critical systems.

CCS CONCEPTS

• **Computing methodologies** → **Graphics processors**; • **Computer systems organization** → **Heterogeneous (hybrid) systems**; • **Software and its engineering** → **Parallel programming languages**; • **Hardware** → **Safety critical systems**.

KEYWORDS

Safety Critical Systems, Embedded Multicores, Embedded GPUs, SYCL

ACM Reference Format:

Cristina Peralta Quesada, Matina Maria Trompouki, and Leonidas Kosmidis. 2023. Evaluation of SYCL's Suitability for High-Performance Critical Systems. In *International Workshop on OpenCL (IWOCL '23)*, April 18–20, 2023, Cambridge, United Kingdom. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3585341.3585378>

1 INTRODUCTION

In safety critical systems, like the ones used in automotive, avionics and aerospace, both correct and timely execution are of paramount importance, because their malfunction can result in harm of human lives. Traditionally these systems have been using very simple processing technologies such as single core, in-order processors.

However, the recent market advances in safety critical systems require advanced functionalities which cannot be provided with these processors. State-of-the-art cars include several advanced driving assistance systems (ADAS) such as autonomous parking, heads-up (HUP) windscreen displays and smart mirrors [26]. Some of these features are also mandatory for new models, such as the automatic emergency breaking system which is necessary for all cars sold in the European Union starting from 2022 [20]. More ADAS functionalities are going to be gradually mandated in the following years, while all car manufacturers work intensively towards the implementation of fully autonomous cars.

Other safety critical industries face similar requirements. In avionics, Airbus recently performed test flights for their Autonomous Taxi, Take-Off and Landing (ATTOL) project [1] as well as vision-based automatic air-to-air-refueling[2]. In the space domain, there is also a trend towards increased autonomy of spacecraft and a shift of operations which in the past used to take place on ground, on-board. For example, the European Space Agency (ESA) Φ-Sat-1 technology demonstration mission [6] has been the first earth observation cubesat which demonstrated the use of artificial intelligence on-board a space system, in order to avoid transmitting to the ground earth images which have been covered by clouds.

Similar functionalities have been also included in the ESA OP-SAT cubesat [7], while the soon to be launched Φ-Sat-2 will have more advanced functionalities. Other space agencies explore also autonomous operations such as the recent successful DART mission (Double Asteroid Redirection Test) [15] from NASA, which was using vision based navigation and was operating completely autonomously during the last couple of minutes before impact.

All these advanced functionalities require very high computational power, so all safety critical domains are looking for new candidate platforms.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

IWOCL '23, April 18–20, 2023, Cambridge, United Kingdom

© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0745-2/23/04...\$15.00
<https://doi.org/10.1145/3585341.3585378>

Systems-on-chip (SoCs) featuring multicores and embedded Graphics Processing Units (GPUs) are considered very attractive candidate platforms [13][5][12] and several promising case studies have been demonstrated on them for all these sectors. However, programming these platforms is challenging. An important factor for their adoption in the safety critical industries is not only their peak computational capability, but also the ease of their programming, and specifically the trade-off between programmability and the achieved performance. Prior works have evaluated the computational capabilities of such platforms, comparing the multicore CPU and GPU computational capabilities [12] using the OpenMP and CUDA programming models respectively.

Recent works argue towards the adoption of high level programming models in safety critical systems [22]. In particular, Khronos is currently investigating with an Exploratory Group the introduction of SYCL SC (Safety Critical), a subset of the SYCL API which will target safety critical systems. For this reason, in this paper we focus on SYCL to evaluate its potential for future adoption in embedded multi-core and GPU platforms for safety critical systems.

For our evaluation, we rely on open source software for two safety critical domains, aerospace and automotive which we port in SYCL. We compare its performance and programmability with other parallel programming models on two multicore and GPU platforms, one from the high performance domain and one specifically focusing on the automotive domain. Our results indicate that in many cases, SYCL can provide comparable performance to highly optimised code using CUDA or NVIDIA libraries with a lower development effort and complexity, which shows the suitability of SYCL for the development of high-performance safety critical software.

2 HIGH-PERFORMANCE SAFETY-CRITICAL SOFTWARE SELECTION

Since the focus of this work is on safety critical systems, the selected software needed to be related to at least one safety critical industry and computationally intensive, so that it requires multicore and GPUs in order to be accelerated. Additionally, the software needed to be already available for other parallel programming models, so that a comparison with them would be possible without the need to port it again.

The only open source software satisfying these requirements was the open source GPU4S Bench benchmarking suite [18], developed at the Barcelona Supercomputing Center (BSC) within the GPU4S (GPU for Space) project [13] funded by the European Space Agency (ESA). GPU4S Bench consists of multiple algorithmic building blocks commonly used in the space domain and in other safety critical systems. However, in order to have a holistic evaluation we included a full application for safety critical systems, an open source Pedestrian Detection application [23] for NVIDIA GPUs.

Therefore, with our software selection we cover two safety critical domains - aerospace and automotive - and two types of software, general algorithmic building blocks and a full application.

2.1 GPU4S/OBPMark Kernels Benchmark Suite

Due to the lack of open source space domain benchmarks available for GPUs, the GPU4S Bench [18] was developed at BSC with the objective of developing an open source, space-relevant benchmark

suite with implementations in different GPU programming models, aiming to evaluate the potential of embedded GPUs in space domain. Known also as OBPMark Kernels, GPU4S Bench is part of the ESA OBPMark (On-Board Processing) Benchmark suite [21], which is hosted in GitHub under an ESA GPLv3 compatible license [10]. OBPMark is the official ESA benchmarking suite used for all new ESA projects, not only for evaluating GPUs, but also for other high performance on-board processing devices such as multicores, Digital Signal Processors (DSPs), FPGAs and others, replacing prior proprietary solutions. This enables direct comparison of different processing technologies for a variety of on-board processing tasks and reproducibility for the selection of the most appropriate processing solution for a given space mission, without intellectual property and export restrictions which dominate the space sector.

GPU4S Bench [10] is designed to evaluate the performance of parallel embedded GPUs and multicores in the space domain, as well the programmability of their programming models. It aims to cover as many space domains as possible, for this reason, it includes a wide variety of common algorithms encapsulated in building blocks which appear in different contemporary and future space software. The selection of the algorithmic building blocks has been performed based on survey performed by the GPU4S project partner Airbus Defence and Space, across all space software divisions of the company [18]. This allows the evaluation of multiple space domains with low effort.

While GPU4S Bench was initially targeting GPUs, the lack also of multicore benchmarks for safety critical systems led to porting them also in OpenMP [12]. GPU4S Bench includes three implementations (naive, optimised and vendor library) per algorithmic building block, each of which features different optimisation levels and vendor library usage. These different versions are developed to compare the performance against, in theory, optimal vendor developed versions of the algorithms and the ease of implementation versus the achieved performance on naive and optimised versions. The reason for this is that it is not only important to consider the obtained performance, but also the effort to achieve such performance. For example it may be convenient in some occasions to utilise a naive approach as long as its performance is acceptable, especially compared to a legacy space processor.

The latest version of GPU4S Bench [10] includes implementations in CUDA, HIP, OpenMP and OpenCL. All benchmarks also have a reference sequential version which is used for the verification of the various implementations.

In this paper we ported GPU4S benchmarks to SYCL using the two memory models, Unified Shared Memory (USM) and Buffers. Our implementation follows the benchmark organisation and the building system of the GPU4S Benchmarking suite [18]. The ported code for SYCL is provided in separate directories and the algorithmic building blocks are invoked from common functions which are calling the appropriate parallel programming model implementation. This guarantees that all programming models are using the same driver code and therefore have the same overheads. The selection of the programming model is performed at compilation time, using preprocessor directives.

In order to be able to choose the device in SYCL (multicore or GPU) we included a `device_selector` function, which is used later to create the appropriate queue where SYCL kernels are launched.

Our implementation is available at [16] and will be submitted for inclusion in the next release of the GPU4S Bench repository in GitHub [10].

2.2 Pedestrian Detector

As a representative of a full automotive application, we used a pedestrian detection application for GPUs. This open source software was developed as a multi-CPU multi-GPU benchmark for automotive systems, achieving an 88× speedup compared to the sequential version of the same algorithm running on a server class CPU, by using 4 CPUs and 2 GPUs in a high performance computing system [23]. In this way, a real-time processing of 21 frames per second was achieved.

This application implements a functionality which is found in modern cars for the implementation of the automatic emergency breaking ADAS feature, which is mandatory since 2022 in all new cars sold in the European Union [20]. The detection is based on the Viola-Jones [24][25] method. While the accuracy of this conventional vision method is lower than solutions based on deep neural networks, it has the advantage of being explainable, which is very important for the certification of such algorithms in systems that are subject to this process, as it is the case of automotive systems.

The application takes as an input a classifier file, containing the parameters of the features used for the detection (i.e. equivalent to a deep neural network model in Deep Neural Network (DNN) inference applications) and a dataset of black and white images which resemble images obtained from a vehicle camera. In our case these are .pgm images with a size of 480x640 pixels. The software detects pedestrians on the images, highlighting the detection in the output images, and also prints their coordinates. This information can be used by a subsequent automotive system to implement emergency breaking.

The application consists of three main tasks. The first task is equivalent to image acquisition from the camera. The second task, which is the most computationally intensive, detects the pedestrians by comparing the pixel values of sections of the image to a threshold defined by a cascade of a hierarchically arranged set of weak classifiers, which detect certain features in these areas. This operation is performed multiple times on various window sizes. The third task filters the sections of the detected pedestrians (non-maximum suppression), highlights the detections on the output image and saves it to a file. This might be needed for post-accident examination for legal or insurance-related reasons. The application performs the detection on one image at a time.

The original application features a highly optimised multi-CPU multi-GPU implementation in a combination of MPI and CUDA. Moreover it achieves high performance by overlapping GPU transfers and computations. Note that unlike GPU4S Bench, no OpenMP implementation is available, so this application is only used to evaluate the performance of the SYCL implementations on the GPU.

For the SYCL porting we used the CUDA implementation as a starting point. However, the biggest difference is that the code needed to be ported first to C++, and convert the CUDA kernels to SYCL ones. Again we produced two versions of the code, one with the USM and one with the Buffer memory model. Our implementations are available in [17] and are going to be submitted for inclusion in the official GitHub repository [14].

3 PROGRAMMABILITY AND SAFETY CRITICAL CODING GUIDELINES CONSIDERATIONS

SYCL offers two memory management models, USM and Buffers [8]. Based on our experience during this work, when the development starts from zero, the Buffers memory model is easier to use as the programmer does not need to worry about data transfers nor dependencies between kernels. This is a major boost of programmability, since it keeps the code complexity low, which is important for safety critical systems certification [11][9][5]. However, the downside of Buffers is that for a programmer already familiar with other lower level GPU programming models (CUDA, OpenCL etc.), this memory model does not offer any advantage over USM, as its performance is expected to be lower, which is consistent with our performance results. On the other hand, USM uses pointers and dynamic memory allocations, which are discouraged for use in safety critical systems [4][9]. However, a major improvement of SYCL USM in this aspect compared to other low level GPU programming models, is the ability to use automatically computed indices e.g. in `parallel_for` constructs, which are correct-by-construction similar to [22]. This reduces programming mistakes and makes SYCL USM better for use in safety critical systems.

Another important consideration for the two memory models is the low performance when the Buffers' accessors are not declared correctly, so small mistakes might lead to many non-required data transfers. SYCL USM is a better approach as, in our experience, it delivers similar performance to CUDA's programming model. At the same time, it offers the SYCL advantages which make coding faster, such as the possibility to create simple kernels with `parallel_for` or the availability of in-order queues and easy to use kernel and data management tools for asynchronous kernels.

In terms of lines of code (LOC), the Buffer model is longer than USM as any kernel launch requires the creation of accessors for the data used inside the kernel. On the other hand, the lines of code required for the USM model is in the same range with the lines of code required for the CUDA implementations.

In comparison to OpenMP, SYCL has higher complexity, since it requires code modifications instead of source code annotations with pragmas. However, once the software is ported to SYCL, in both programming models the same code base can be used to run in a multi-core system or GPU, and requires similar low maintenance effort. In addition, the obtained multi-core performance is similar to the OpenMP.

Overall, SYCL provides higher programmability and lower complexity compared to low-level GPU programming languages, which makes it more attractive for safety critical systems. From the two memory models, USM offers a reasonable trade-off among compliance with safety critical guidelines, complexity and performance. Finally, SYCL is a good alternative to OpenMP for multi-core safety critical systems.

4 EXPERIMENTAL SETUP

To the best of our knowledge SYCL had never been demonstrated on an embedded platform, so the code was first ported to SYCL in a high performance server platform, which was known that SYCL could be supported.

Benchmark	LRN	Conv.2D	Softmax	FFT	Matrix mult.	Max pooling	ReLU	Wavelet
Input size	2048	2048	8192	262144	1024	16384	8192	2097152
Filter Size	3			64				

Table 1: Configuration parameters for all GPU4S Bench building blocks on the high performance x86_64 server.

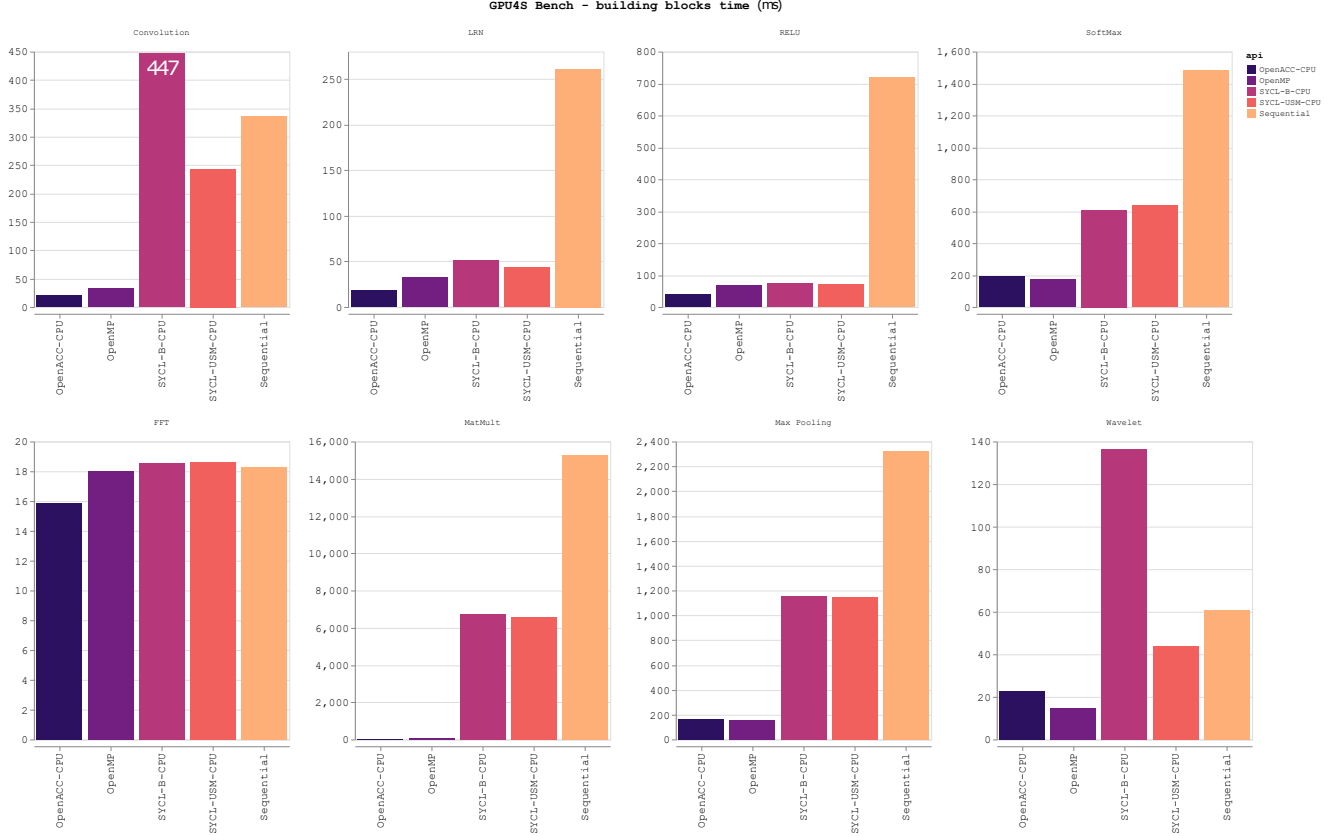


Figure 1: Execution time (ms) of GPU4s building blocks for CPU parallel programming models on the high-performance x86_64 server.

As a high performance development platform we have used an x86_64 server with an AMD Ryzen 7 1800 Eight-Core Processor and 16 hardware threads, equipped with an NVIDIA GeForce GTX 1080 Ti GPU. The system is capable of full duplex transfers over PCIe, both from and towards the GPU at the same time. The server uses Ubuntu 18.04.6 LTS, gcc 7.5 and supports CUDA 11 and OpenMP 4.5. In order to install SYCL in this platform, we built the latest available hipSYCL (v0.9.3) from sources [3].

As an embedded high-performance platform representative of the safety critical systems we used NVIDIA Xavier AGX. This embedded platform is designed to comply with the automotive safety standard ISO 26262 [11] and is also considered a good candidate for use in space [19]. It is a heterogeneous embedded platform featuring a 8-core Carmel ARM v8.2 64-bit CPU, a Volta GPU with 8 SMs (Streaming Multiprocessors) and 32GB of LPDDR4 memory. Our platform runs Ubuntu 18.04 with Linux kernel version 4.9.253, gcc 7.5.0 with OpenMP version 4.5 and includes CUDA 10.2. This

platform has shared memory between the CPU and the GPU, and it is only capable of a single memory transfer from or towards the GPU memory at a time, since it contains only a single copy engine.

NVIDIA provides various power modes for the Xavier AGX platform with a Thermal Design Power (TDP) ranging from 10W to 30W. For the evaluation performed in our work we have chosen the power mode 2 (15W), as it is the maximum allowed power budget that has been identified in the GPU4S project due to heat dissipation limitations in space [13]. At 15W, power mode 2 offers 4 CPU cores running at 1200MHz and the 8 SMs of the GPU running at 670MHz. Therefore, this power mode provides a balance between the available CPU cores and the GPU, which is appropriate for a fair comparison of the CPU and GPU implementations of the programming models we examine in this paper.

In order to enable the use of SYCL, we compiled hipSYCL its source code repository [3] as we did in the high performance x86_64 server, by setting CUDA as a backend.

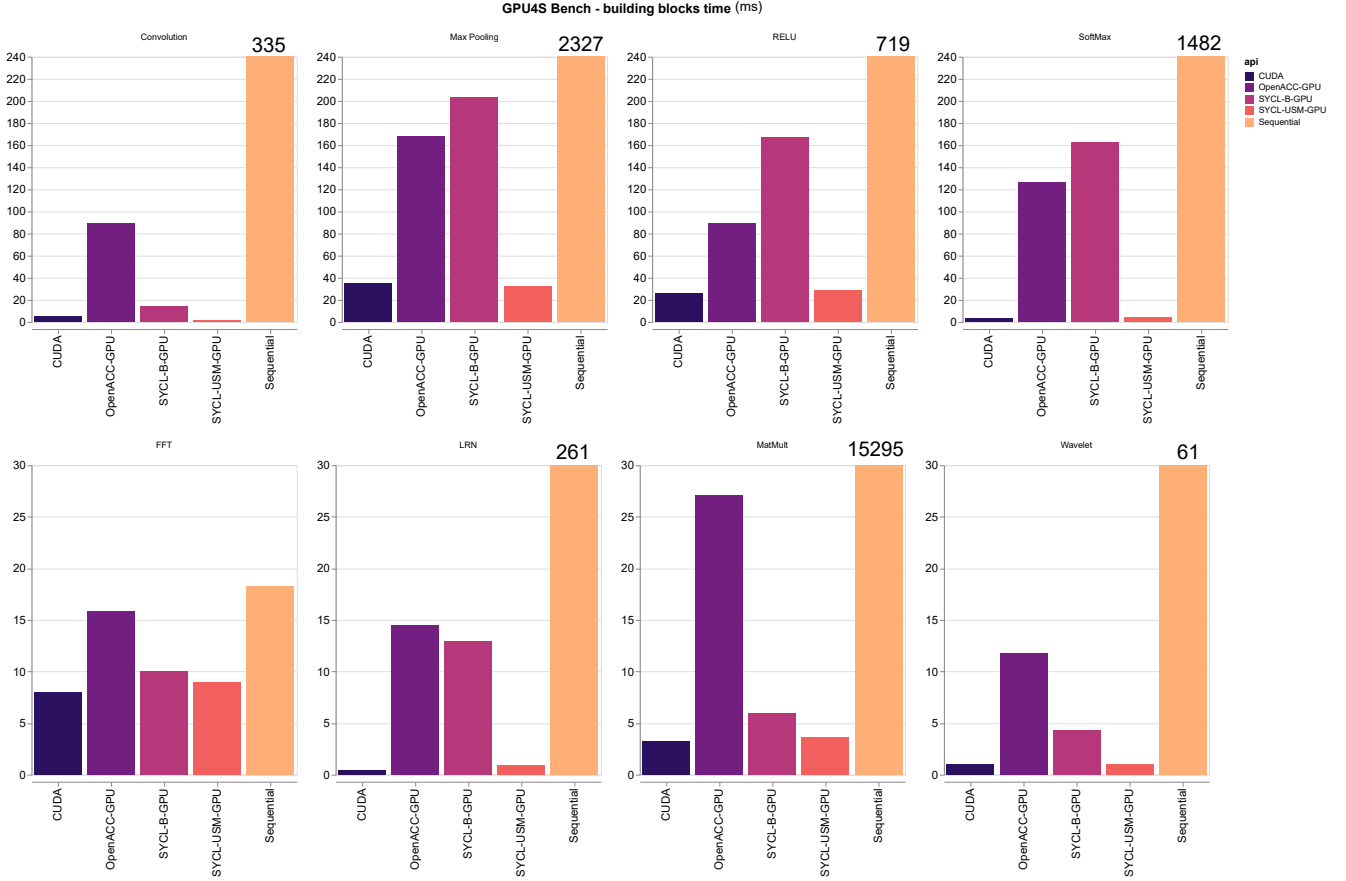


Figure 2: Execution time (in ms) per building block for parallel GPU programming models on the high-performance server.

5 EVALUATION

5.1 GPU4S Benchmarks

In this section we present the experiments we performed in order to evaluate the performance of the developed implementation of GPU4s Bench benchmarks in SYCL, OpenMP and CUDA.

5.1.1 High performance x86_64 server.

In order to run our tests on a high performance x86_64 server we selected the configuration parameters shown in Table 1 for each of the selected building blocks. The configurations include the input sizes of each benchmark, as well as the filter size for the convolution. Moreover, the GPU implementations of the GPU4S Bench are written in CUDA and OpenCL which are low level GPU programming models and therefore the programmer has to select the block size of the GPU kernels. The selected block size is very important for the obtained performance. For this reason, we have performed an exploration of the block size that gives the best performance in CUDA for the most computationally intensive GPU4S Benchmarks, i.e. the matrix multiplication, the FFT and the convolution.

Comparison with OpenMP: To begin with the CPU evaluation we determined the best number of threads to be used by the OpenMP versions using the `OMP_NUM_THREADS` environment

variable. For each of our selected benchmarks' building blocks we evaluated the programs for 4, 8, and 16 threads for a fixed input size and we select the best performing one. Since SYCL does not allow thread number customisation, the runtime performs this selection.

Figure 1 shows the execution times of every building block for Sequential, OpenMP, SYCL USM and SYCL Buffer (shown as SYCL B) versions with their best performing number of threads configuration. We notice that both SYCL variants are outperformed by OpenMP. We suspect that this difference is in part due to the inability to set the number of CPU threads in SYCL. Otherwise, given that our SYCL implementation, hipSYCL uses OpenMP as backend, we did not expect that difference.

Moreover, we notice that in general the Buffer implementation of SYCL is slower than the USM version. This is expected, because the automatic memory management performed by SYCL when Buffers are used, is usually less efficient than the explicit memory management which is performed with USM. In addition, it is important to consider that our SYCL code is GPU-oriented. This means that although we are able to execute it in any multicore CPU thanks to SYCL's OpenMP backend, it is suboptimal. Parts of the code for GPU optimised versions cannot be run efficiently on CPU, such as any code using barriers. This is a known performance issue of

Benchmark	LRN	Conv.2D	Softmax	FFT	Matrix mult.	Max pooling	ReLU	Wavelet
Input size	2048	1024	8192	32768	1024	4096	8192	2097152
Filter Size	3			64				

Table 2: Configuration parameters for all GPU4S Bench building blocks on the NVIDIA Xavier.

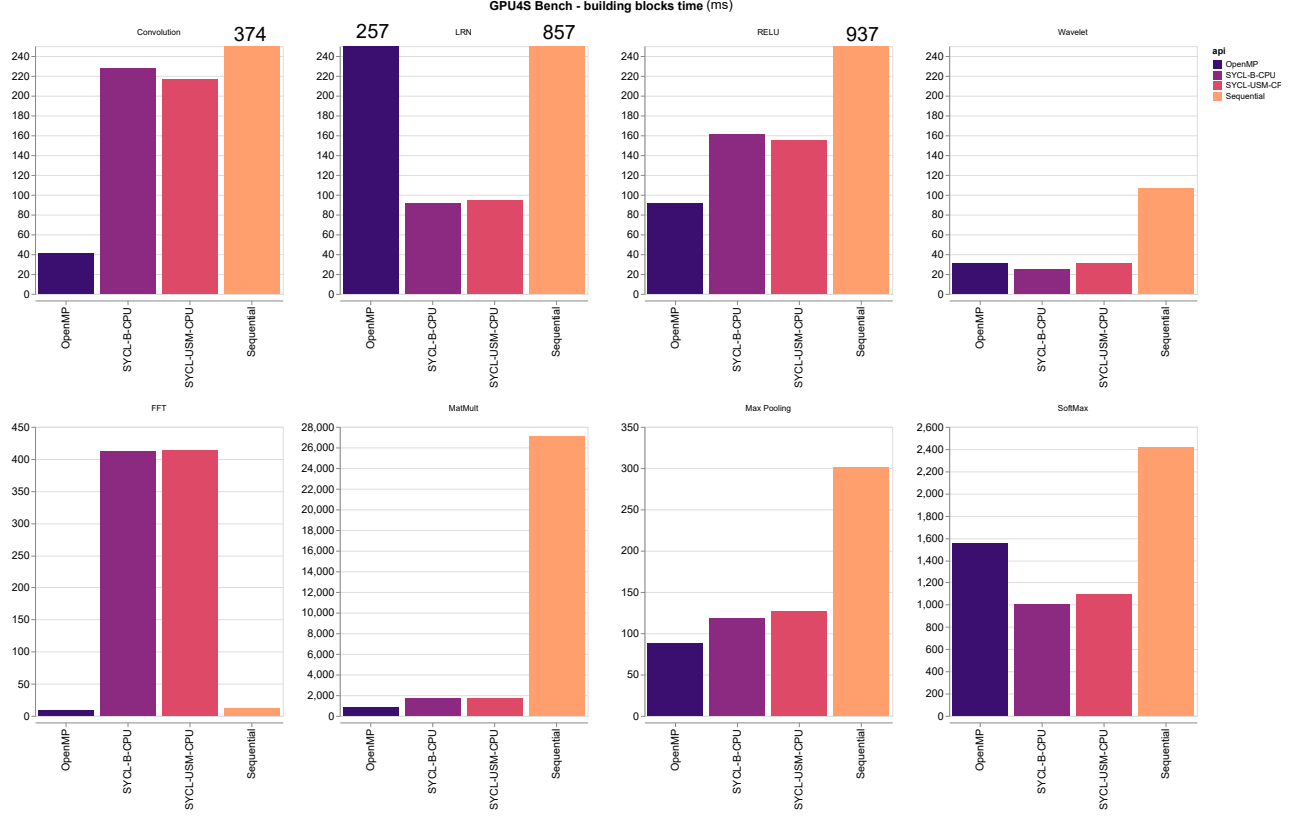


Figure 3: CPU Execution Times (in ms) for GPU4S Building Blocks on the NVIDIA Xavier.

the SYCL CPU backend and specifically on our hipSYCL version. Therefore, the presented results are with the best performing kernel implementations, which are the naive versions. However, we did not consider providing another optimised version for the SYCL CPU backend, since we believe that the GPU can provide much higher computational capabilities for safety critical systems compared to multi-cores. This is evident by comparing our multicore and GPU implementations, and supported also by other works in the literature [12].

Comparison with CUDA: We first tested our building blocks for block sizes of 4x4, 8x8, 16x16 and 32x32 for CUDA and SYCL and we selected the best performing ones. Not all optimised versions of SYCL require to define a block size, but shared memory and synchronisation between work items is only allowed inside `nd_range` kernels. The size of the kernel needs to be set on `nd_range` kernel definition. For this reason we decided to take the same approach as

on CUDA code, hence CUDA and SYCL optimised kernel versions have the same `nd_range` and block size.

Figure 2 shows the execution time results using the best block sizes for each building block. In general we can see that CUDA provides the best performance, as expected, but the SYCL USM version provides a very similar performance, which means that it provides a good trade-off between programmability and performance. On the other hand, SYCL Buffers version is slower than USM as expected.

5.1.2 NVIDIA Xavier.

Similarly we first defined the configuration of the GPU4S Bench we used, which is shown in Table 2. Note that this is an embedded platform with significantly less computational power compared to the high performance x86_64 server. For this reason, the input sizes have been reduced compared to Table 1 in order to have reasonably long execution times.

Comparison with OpenMP: In Figure 3 we present the execution times for our selected building blocks on the NVIDIA Xavier.

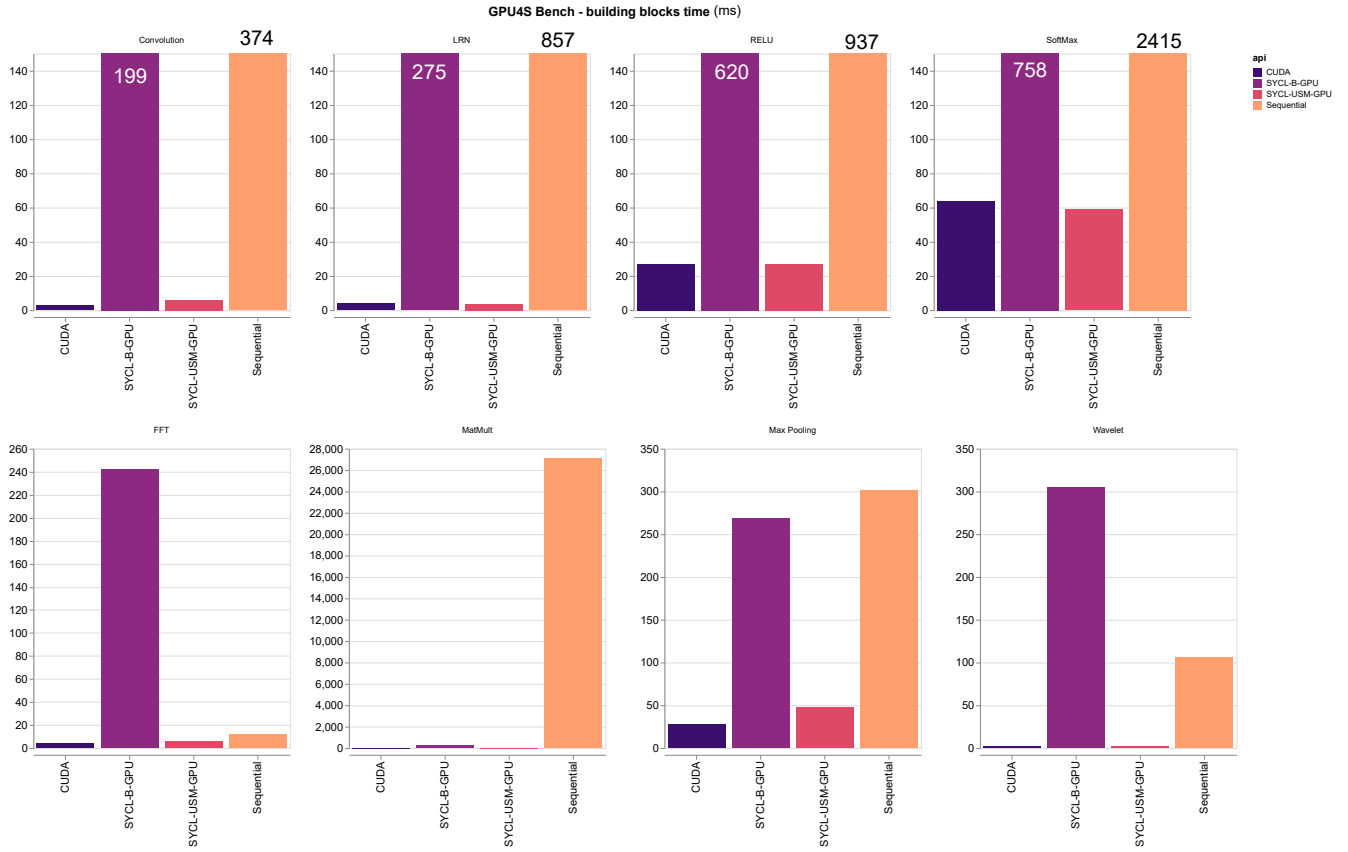


Figure 4: Execution time (in ms) per building block for parallel GPU programming models on NVIDIA Xavier.

Again, the SYCL Buffer implementation provides lower performance than OpenMP and SYCL USM. However, unlike the high performance x86_64 server, in some benchmarks we notice a very interesting behavior, that the SYCL implementation is faster than the OpenMP implementation. This is the case of convolution, local response normalisation (LRN), softmax and wavelet transform. These building blocks are frequently found in neural network inference applications and are the least computationally intensive among the GPU4S Bench algorithms. Therefore their thread parallelisation overhead on a less power full CPU like the one found in the NVIDIA Xavier can be more pronounced. This is because the AMD CPU of the high performance x86_64 server is heavily multithreaded and highly optimised for single thread performance, while the ARM Carmel CPU of the NVIDIA Xavier is not multithreaded and has significantly lower single core performance. Our hypothesis is that SYCL USM is able to optimise the code of these short kernels using SIMD vectorisation in the form of ARM NEON instructions, and therefore avoiding the overhead of spawning new threads which is the case in OpenMP.

Comparison with CUDA: As in the high performance server, we started by looking for the best block size configuration of CUDA and SYCL, ranging from 4x4, 8x8, 16x16 and 32x32 blocks on all the building blocks.

In Figure 4 we present the GPU execution time on Xavier. We observe the same trends as in the high performance server platform. CUDA provides the best performance, while the SYCL USM version has very similar performance. Again SYCL B is the slowest, this time with a broad margin. In fact, for some benchmarks like the FFT and the wavelet transform, the SYCL Buffer implementation is even slower than the sequential version. Although it was expected due to memory transfers, our analysis showed that the memory transfers in CUDA and SYCL USM are not taking large part of the overall execution time. Therefore the SYCL B overhead can either be related to the inefficient memory management from the runtime, or suboptimal code generation for the kernel, or both.

Figure 5 shows the comparison of the GPU programming models over different input sizes of the two most computationally intensive benchmarks, the matrix multiplication and FFT. In the case of matrix multiplication, the NVIDIA library implementation, cuBLAS, provides the best performance for the three examined input sizes (1024, 2048 and 4096). For the two smaller input sizes the vendor optimised library version and the SYCL USM implementations provide similar performance, however for the 4096 input size, the NVIDIA library is faster.

In the case of the FFT, the highest performance is achieved by the SYCL USM version, which is marginally faster than the NVIDIA

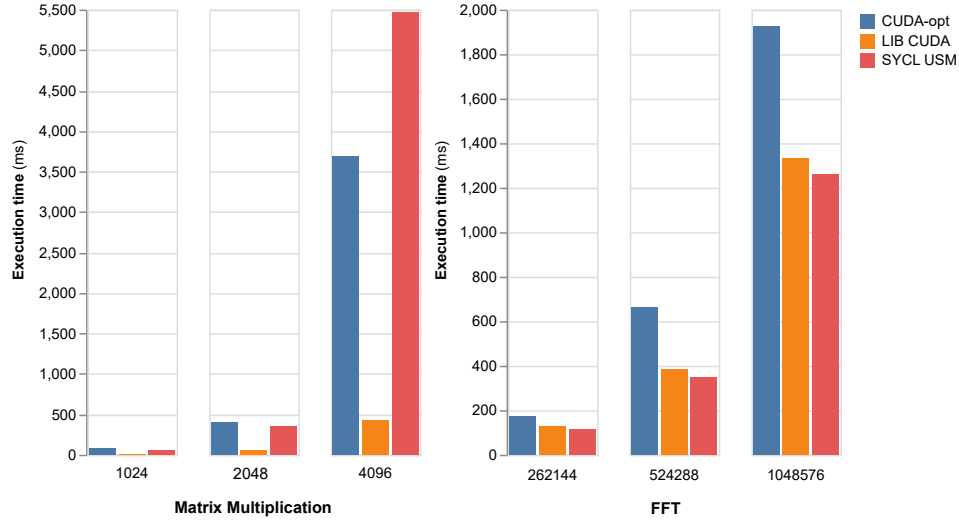


Figure 5: Comparison of SYCL against NVIDIA libraries on various input sizes of matrix multiplication and FFT on Xavier.

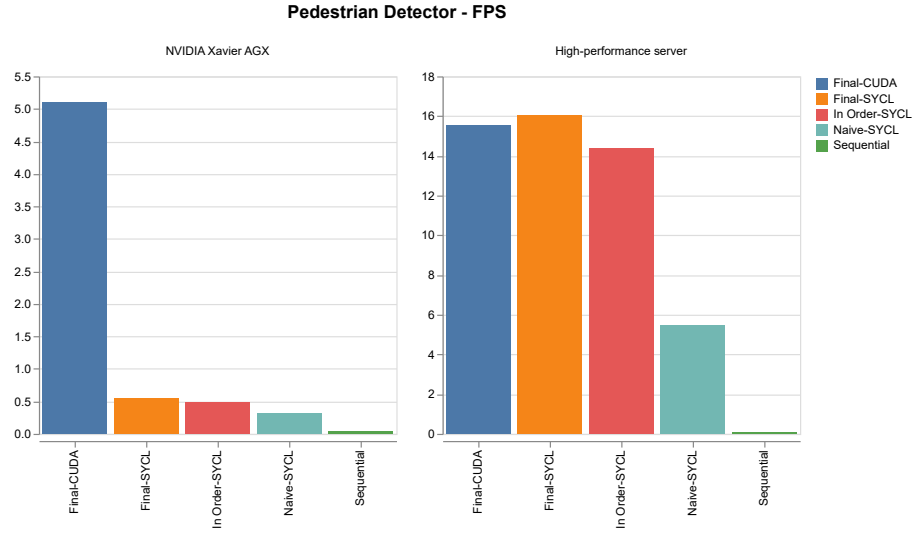


Figure 6: Frames Per Second (FPS) on NVIDIA Xavier AGX (left) and high-performance server (right) for the 3 SYCL implementations of the Pedestrian Detector with the CUDA and sequential versions of the application.

library implementation (cuFFT). The vendor optimised implementation is slower, which means that SYCL USM is able to provide not only higher programmability, but also very high performance compared to a vendor optimised library.

5.2 Pedestrian Detector

We developed 3 versions for SYCL for the Pedestrian Detector. A naive implementation, a second version using in-order SYCL queues, and one with asynchronous out-of-order queues. Next we compare

them against the sequential and the single-CPU, single-GPU CUDA implementation of [23].

Naive Implementation: The main loop of the application is the most time consuming - and computational demanding - task of the code, which checks whether a part of the image contains a pedestrian. In order to parallelise this code we divided the loop into two smaller tasks, one for finding out the points where a pedestrian is detected and another one for the non-maximum suppression of the bounding box.

In Figure 6 we can see the *Frames Per Second* (FPS) performance of the application. The naive version of the SYCL code reaches 5.44

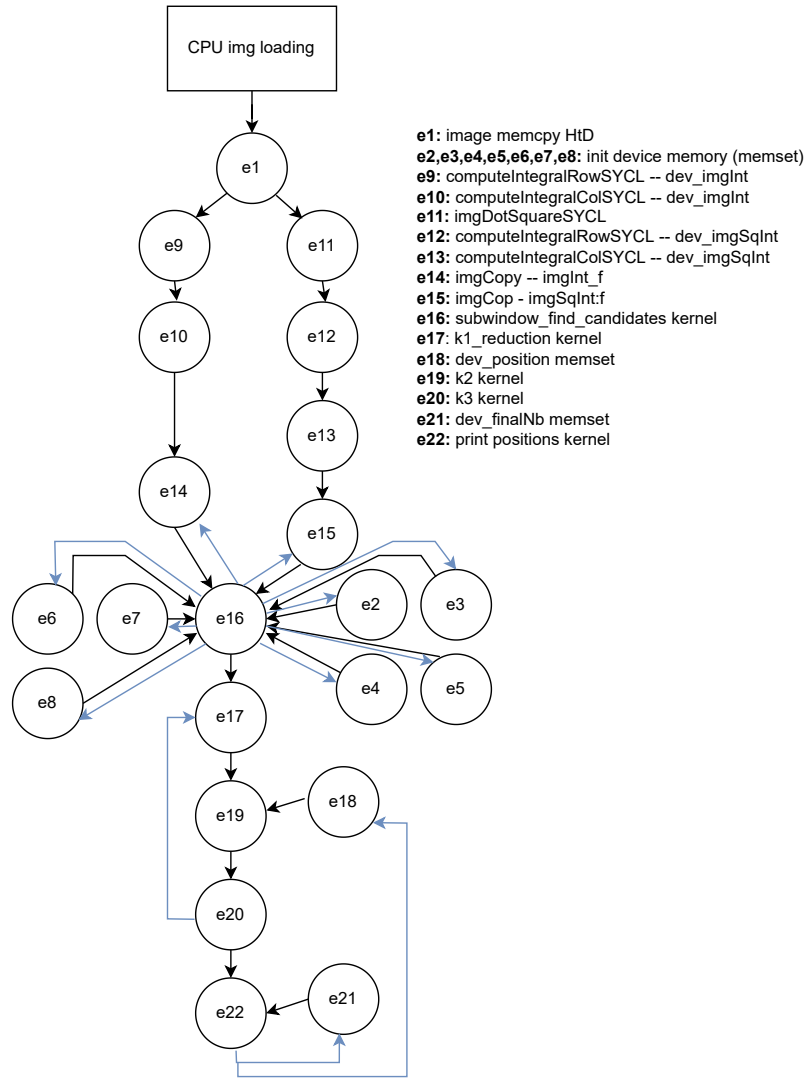


Figure 7: SYCL data flow graph for the pedestrian detector.

FPS, far from the real-time processing of 15 FPS, but significantly better than the 0.88 FPS of the sequential high performance x86_64 server execution. The FPS improvement is not very high in NVIDIA Xavier though, passing from 0.04 FPS for sequential to 0.31 FPS for the naive version.

In-order queue implementation: This version of the code reaches 13.9 FPS on the high-performance server which is close to the 15 FPS goal. Interestingly, this is slightly lower than the performance of the manually optimised CUDA implementation [23] on the same platform, with significantly less optimisation effort. However, for NVIDIA Xavier the performance is limited to less than half FPS (0.49 FPS). Moreover, in the NVIDIA Xavier, the obtained performance is 10× slower than the manually optimised CUDA code of the application. Figure 6 provides a comparison among all the implementations.

We can conclude that SYCL in-order queues offer a great advantage on programmability, since we do not need to worry about kernel dependencies and we can avoid the hassle of defining a data flow graph. However, this comes at the cost of not being able to run parallel streams when using in-order queues. For this reason we decided to develop a final version which can get closer performance to CUDA's implementation.

Final version: For our final version of SYCL code we used out-of-order queues, which allow for parallel stream execution whenever it's allowed on the device. We defined the kernel and data dependencies of the program in a SYCL data flow graph, using SYCL events. Figure 7 shows the kernel and data dependencies of our program, where each node represents a kernel or data transfer operation (identifiable on source code by their event number) and directed arrows represent dependencies (on a single direction). Dependencies coming from a previous iteration are marked as blue.

In Figure 6 we present the results for all the versions of the code, including the final version. We notice that the performance of this version of the code surpasses the 15 FPS real-time processing goal on the high performance server, reaching 16.02 FPS which is slightly faster than the FPS performance of the manual optimised CUDA implementation (15.55 FPS). Note that this is similar to the behavior we have noticed also on some GPU4S Benchmarks but on the NVIDIA AGX Xavier. This shows that despite being a high level programming model, SYCL is possible to achieve performance close or higher compared to lower-level GPU programming models.

On the other hand for NVIDIA Jetson AGX Xavier the performance of our code is far behind CUDA's, as we stay around 0.54 FPS while on CUDA 5 FPS are reached. We suspect that the main performance benefit of the SYCL implementations in high performance systems comes from the exploitation of asynchronous operations, which allow the overlapping of GPU kernel executions and memory transfers. In fact, according to [23], this was one of the most important optimisations performed in the application.

However, embedded GPUs like the NVIDIA AGX Xavier share the same memory with the CPUs and have only a single copy engine. This limits the benefits of this optimisation. Moreover, probably the SYCL generated code is not able to provide the same level of optimisation as the manually written one. For example, in addition to asynchronous operations, the original code relies on the Zero-Copy optimisation using pinned memory which is not exposed to SYCL.

6 CONCLUSION AND FUTURE WORK

In this work we evaluated the suitability of SYCL for multi-core and GPU programming in safety critical systems for the first time, by porting two types of relevant safety critical GPU software, from the aerospace and the automotive domain.

Our target embedded multicore and GPU platform was NVIDIA Xavier which is a candidate platform for use in safety critical systems, both for aerospace and automotive applications, as well as a high-performance server used mainly for development. Our results indicate that the SYCL USM programming model performs significantly better than the SYCL Buffer memory model. Interestingly, we encountered several cases in which the SYCL USM implementation provided the same or marginally faster execution time than hand optimised CUDA code. Considering that a) the amount of code required for the SYCL version is less than the CUDA one, b) significantly less effort is required for the SYCL implementation, c) SYCL provides a less error prone abstraction for safety critical software development and d) the fact that the SYCL code is portable among devices from different vendors, we conclude that SYCL provides a good trade-off of programmability and obtained performance. Therefore, we can confirm that the on-going work in Khronos for the definition of the Khronos SYCL SC working group is a step towards the right direction.

However, we have noticed that while SYCL provided very similar performance to the CUDA version for the pedestrian detection application in the high performance server platform, its performance on the NVIDIA Xavier was 10× slower. This is something that needs more investigation in order to identify whether the reason of this difference is the hardware differences of the embedded platforms,

or the inability of the current version of our SYCL implementation (hipSYCL v0.9.3) to take advantage of these differences. For this reason, in the future we plan to evaluate other SYCL implementations such as the one provided by Intel. Finally, in the future we want to evaluate Intel's DPC++ Compatibility Tool and compare its performance and programmability against the manually performed code porting we did in this work.

ACKNOWLEDGMENTS

This work was funded by the Ministerio de Ciencia e Innovacion - Agencia Estatal de Investigacion (PID2019-107255GB-C21 and IJC-2020-045931-I MCIN/AEI/10.13039/501100011033), the European Commission's Horizon 2020 programme under the UP2DATE project (grant agreement 871465) and the HiPEAC Network of Excellence.

REFERENCES

- [1] Airbus. 2020. *Airbus Demonstrates First Fully Automatic Vision-based Take-off*. <https://www.airbus.com/en/newsroom/press-releases/2020-01-airbus-demonstrates-first-fully-automatic-vision-based-take-off>.
- [2] Airbus. 2022. *A330 MRTT becomes world's first tanker certified for automatic air-to-air refuelling operations*. <https://www.airbus.com/en/newsroom/press-releases/2022-07-airbus-a330-mr-tt-becomes-worlds-first-tanker-certified-for>.
- [3] A. Alpay and V. Heuveline. 2020. SYCL beyond OpenCL: The Architecture, Current State and Future Direction of HipSYCL. In *Proceedings of the International Workshop on OpenCL (Munich, Germany) (IWOCL '20)*. Article 8.
- [4] Motor Industry Software Reliability Association. 2013. *MISRA-C-2012. Guidelines for the Use of the C Language in Critical Systems*. MISRA.
- [5] M. Benito, M. M. Trompouki, L. Kosmidis, J. D. Garcia, S. Carretero, and K. Wenger. 2021. Comparison of GPU Computing Methodologies for Safety-Critical Systems: An Avionics Case Study. In *Design, Automation & Test in Europe Conference (DATE)*.
- [6] European Space Agency. 2020. *Φ-sat Artificial intelligence for Earth observation*. https://www.esa.int/Applications/Observing_the_Earth/Ph-sat.
- [7] F. Férèsin, E. Kervennic, Y. Bobichon, E. Lemaire, N. Abderrahmane, G. Bahl, I. Grenet, M. Moretti, and M. Benguigui. 2021. In Space Image Processing Using AI Embedded on System on Module: Example of OPS-SAT Cloud Segmentation. In *2nd European Workshop on On-Board Data Processing (OBDP)*. <https://doi.org/10.5281/zenodo.5574960>
- [8] Khronos Group. 2022. SYCL 2020 Specification. <https://registry.khronos.org/SYCL/specs/sycl-2020/pdf/sycl-2020.pdf>.
- [9] G.J. Holzmann. 2006. The power of 10: rules for developing safety-critical code. *Computer* 39, 6 (2006), 95–99.
- [10] L. Kosmidis D. Steenari I. Rodriguez, A. Jover. 2020. GPU4S Bench Code Repository. https://github.com/OBPMARK/GPU4S_Bench.
- [11] International Organization for Standardization. 2018. *ISO 26262. Road Vehicles – Functional Safety*.
- [12] Á. Jover-Alvarez, A. J. Calderón, I. Rodriguez, L. Kosmidis, K. Asifuzzaman, P. Uven, K. Grüttnert, T. Poggi, and I. Agirre. 2021. The UP2DATE Baseline Research Platforms. In *Design, Automation & Test in Europe Conference (DATE)*.
- [13] L. Kosmidis, I. Rodriguez, A. Jover-Alvarez, S. Alcaide, J. Lachaize, O. Notebaert, A. Certain, and D. Steenari. 2021. GPU4S: Major Project Outcomes, Lessons Learnt and Way Forward. In *Design, Automation & Test in Europe Conference (DATE)*.
- [14] M. M. Trompouki. 2013. *Pedestrian Detection Source Code Repository*. https://github.com/mtrompouki/pedestrian_detection.
- [15] Ned Potter. 2022. NASA's DART Mission Aims to Save the World. <https://spectrum.ieee.org/planetary-protection-nasa-dart>. *IEEE Spectrum* (2022).
- [16] Cristina Peralta Quesada. 2022. *GPU4S Bench SYCL Implementation*. https://github.com/crispq95/GPU4S_Bench.
- [17] Cristina Peralta Quesada. 2022. *Pedestrian Detection in SYCL*. https://github.com/crispq95/pedestrian_detector.
- [18] I. Rodriguez, L. Kosmidis, J. Lachaize, O. Notebaert, and D. Steenari. 2019. *GPU4S Bench: Design and Implementation of an Open GPU Benchmarking Suite for Space On-board Processing*. Technical Report UPC-DAC-RR-CAP-2019-1. Universitat Politècnica de Catalunya. https://www.ac.upc.edu/app/research-reports/public/html/research_center_index-CAP-2019,en.html.
- [19] I. Rodriguez-Ferrandez, M. Tali, L. Kosmidis, M. Rovituso, and D. Steenari. 2022. Sources of Single Event Effects in the NVIDIA Xavier SoC Family under Proton Irradiation. In *2022 IEEE 28th International Symposium on On-Line Testing and Robust System Design (IOLTS)*. 1–7.

- [20] Philip E. Ross. 2022. Brakes that Slam Themselves: Automatic emergency braking will become standard in Europe. *IEEE Spectrum* 59, 1 (2022).
- [21] David Steenari, Leonidas Kosmidis, Ivan Rodriquez, Alvaro Jover, and Kyra Förster. 2021. OBPMark (On-Board Processing Benchmarks) - Open Source Computational Performance Benchmarks for Space Applications. In *European Workshop on On-Board Data Processing (OBDP)*. <https://doi.org/10.5281/zenodo.5638577>
- [22] M. M. Trompouki and L. Kosmidis. 2018. Brook Auto: High-Level Certification-Friendly Programming for GPU-powered Automotive Systems. In *55th ACM/ESDA/IEEE Design Automation Conference (DAC)*.
- [23] M. M. Trompouki, L. Kosmidis, and N. Navarro. 2017. An Open Benchmark Implementation for Multi-CPU Multi-GPU Pedestrian Detection in Automotive Systems. In *2017 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*.
- [24] P. Viola and M. Jones. 2001. Rapid Object Detection Using a Boosted Cascade of Simple Features. In *Proceedings of the 2001 IEEE Computer Society Conference on Computer Vision and Pattern Recognition. CVPR 2001*, Vol. 1. I–I.
- [25] P. Viola and M. Jones. 2004. Robust Real-Time Face Detection. *International Journal of Computer Vision* 57 (2004), 137–154.
- [26] A. O. Yontem, K. Li, D. Chu, V. Meijering, and L. Skrypchuk. 2021. Prospective Immersive Human-Machine Interface for Future Vehicles: Multiple Zones Turn the Full Windscreen Into a Head-Up Display. *IEEE Vehicular Technology Magazine* 16, 1 (2021), 83–92.